	ITER, SIKSHA 'O' ANUSANDHAN (Deemed to be University)		Assignment 4
Branch	**CSE**	Programme	**B.Tech**
Course Name	**Computer Science Workshop 2**	Semester	**4th**
Course Code	**CSE 3141**	Academic Year	**2025-2026**
Assignment: 4	**Topic: Extending Your Blog Application**		
Learning Level (LL)	**L1**: Remembering	**L3:** Applying	**L5**: Evaluating
	L2: Understanding	**L4**: Analysing	**L6**: Creating

Q's	Questions	COs	LL
Q1.	Enhance the blog application by implementing a tagging system using django-taggit. Configure the application, modify the Post model to include tagging functionality, and apply migrations. After implementation, verify that tags can be added and managed through the Django administration interface. **Expected Outcomes** • Tags successfully added to posts • Multiple tags can be assigned to a single post	CO2	**L2, L3**
Q2.	Modify the blog templates to display tags associated with each post. Ensure that tags are displayed in a readable format and converted into clickable links so that users can navigate and view posts filtered by selected tags. **Expected Outcomes** • Tags displayed below each post • Tags shown in link format • Clicking a tag filters related posts	CO2	**L3, L4**
Q3.	Extend the post list view to support filtering of posts based on tags. Update the URL configuration to accept a tag slug parameter and implement logic to retrieve posts associated with the selected tag. Ensure proper handling of invalid tags. **Expected Outcomes** • URL supports tag-based filtering • Posts filtered correctly using selected tag • Invalid tags handled using error response	CO2	**L3, L4**
Q4.	Enhance the post detail page by implementing a feature to display similar posts based on shared tags. Use Django ORM aggregation functions to rank posts according to the number of common tags and display the most relevant results. **Expected Outcomes** • Similar posts displayed below content • Posts ranked based on shared tags • Limited number of relevant posts shown	CO2	**L3, L4**

Q5.	Create a custom template tag library for the blog application. Implement a simple tag to display the total number of published posts and inclusion tags to display the latest posts and most commented posts in the sidebar. **Expected Outcomes** • Custom template tag created and loaded • Total posts count displayed dynamically • Latest and most commented posts shown in sidebar	CO2	**L3, L4**
Q6.	Enhance the blog application by implementing Markdown support for post content. Create a custom template filter to convert Markdown to HTML and ensure proper rendering of formatted content in templates. **Expected Outcomes** • Markdown syntax supported in posts • Content converted into HTML format • Proper rendering of formatted text	CO3	**L3, L4**
Q7.	Implement a sitemap for the blog application to improve search engine visibility. Create a sitemap class for the Post model and configure the sitemap in the project's URL configuration so that all published posts are included in the sitemap. **Expected Outcomes** • Sitemap generated successfully • Accessible via URL (e.g., sitemap.xml) • All published posts included for indexing	CO3	**L3, L4**
Q8.	Extend the blog application by implementing advanced data handling and search features. Install and configure PostgreSQL as the database backend, use fixtures to dump and load blog data, and implement a full-text search engine using Django and PostgreSQL. Additionally, create an RSS feed to provide the latest blog posts to users. **Expected Outcomes** • PostgreSQL configured successfully • Data exported and imported using fixtures • Full-text search retrieves relevant results • RSS feed displays latest blog posts	CO3	**L4, L6**
Q9.	Develop a new Django project named content_project and create an application named contentApp. Design a model such as Article with fields like title, slug, body, and publish date. Implement tagging, pagination, sitemap, and RSS feed features similar to the blog application, ensuring that users can browse and discover content efficiently. **Expected Outcomes**	CO3	**L4, L6**

	• New project and app created successfully • Tagging and pagination implemented • Sitemap and RSS feed integrated • Content navigation works smoothly		
Q10.	Enhance the application by implementing advanced search and data management features. Configure PostgreSQL for full-text search, implement search functionality for articles, and use fixtures to export and import data. Ensure that the system efficiently retrieves relevant results and supports scalable data handling. **Expected Outcomes** • Full-text search implemented using PostgreSQL • Relevant search results displayed • Fixtures used for data backup and restore • Application handles data efficiently	CO3	**L4, L6**
	-END-		

Note:

1. Assignment carries a weightage of 10% **marks out of 100**
2. Only CO1, CO2, and CO3 outcomes are covered.

Course Outcomes	CO1	Remember and identify the fundamental concepts of web development and Django framework.
	CO2	Explain and interpret Django workflows such as authentication, template rendering, and form handling.
	CO3	Apply Django concepts to develop functional web applications.
	CO4	Implement advanced Django features including APIs, payments, internationalization, and real-time systems.
	CO5	Analyze Django applications with respect to caching strategies, API design, data flow, and performance-related design considerations.
	CO6	Analyze and design an end-to-end Django solution including deployment and system integration.